

IMPLANTACIÓN DE APLICACIONES WEB

Instalación y configuración de un servidor de aplicaciones web

ÍNDICE

/ 1. Introducción y contextualización práctica	4
/ 2. Servidor web: Instalación y configuración	5
2.1. Instalación de Apache	5
2.2. Configuración de Apache	5
2.3. Configuración avanzada de Apache	7
/ 3. Procesamiento de código en el servidor	8
3.1. Instalación de PHP	8
/ 4. Caso práctico 1: “Configuramos Apache y PHP para mostrar los errores de código”	9
/ 5. Sistema gestor de base de datos: Instalación y configuración	10
5.1. Instalación de MySQL	10
5.2. Instalación de PHPMyAdmin	11
5.3. Configuración de MySQL	12
5.4. Añadir un nuevo usuario a phpMyAdmin	13
/ 6. Documentación de la instalación	13
/ 7. Procesamiento de código en el cliente	14
7.1. Cliente ftp	14
7.2. Editor de texto	15
7.3. IDE	15

ÍNDICE

/ 8. Seguridad en el servidor Ubuntu	15
8.1. <i>Firewall</i>	15
8.2. Servidor SSH	16
 / 9. Caso práctico 2: “Configuramos el <i>firewall</i> de nuestro sistema”	17
 / 10. Resumen y resolución del caso práctico de la unidad	18
 / 11. Bibliografía	19

OBJETIVOS

Instalar y configurar todo el software necesario para el funcionamiento de un servidor de aplicaciones web.

Establecer y verificar la seguridad de un servidor web.

Documentar los procesos.

/ 1. Introducción y contextualización práctica

En esta unidad aprenderemos a instalar y configurar el *software* necesario para poner en funcionamiento un servidor de aplicaciones web. Además, aprenderemos a asegurar el sistema y documentar los procesos realizados a través de las actividades y casos prácticos.

Durante la unidad, nos centraremos en unas tecnologías concretas, que son las más demandadas y utilizadas en la actualidad por empresas y organizaciones alrededor del mundo **Apache, MySQL y PHP**, entre otras.

A continuación, vamos a plantear un caso práctico a través del cual podremos aproximarnos de forma práctica a la teoría de este tema.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema, encontrarás su resolución en el apartado ‘Resumen y resolución del caso práctico’.



Fig.1. Logo de una infraestructura LAMP con Ubuntu.



Audio intro. “Hola mundo desde nuestro servidor de aplicaciones”

<http://bit.ly/3DVEOoJ>





/ 2. Servidor web: Instalación y configuración

El servidor web es el software encargado de procesar las peticiones del cliente y de generar respuestas en forma de código en un lenguaje web que es interpretado por el cliente para que el usuario vea de manera apropiada una página web.

El principal servidor web, y el más utilizado en el mundo, es el **servidor HTTP Apache**.

2.1. Instalación de Apache

Para instalar el servidor Apache en un ordenador con Ubuntu Server, debemos seguir estos pasos:

1. **Abrir** una terminal y **actualizar la lista de paquetes** disponibles ejecutando el comando:

```
sudo apt update
```

2. **Instalar** Apache ejecutando el siguiente comando:

```
sudo apt install apache2
```

2.2. Configuración de Apache

3. Una vez instalado, debemos **iniciar el servicio de Apache** ejecutando el siguiente comando:

```
sudo systemctl start apache2
```

4. **Para asegurar que Apache se inicie automáticamente** cada vez que el sistema se reinicie, podemos ejecutar el siguiente comando:

```
sudo systemctl enable apache2
```

5. Para **verificar** que el servidor Apache está funcionando correctamente, podemos abrir un navegador web y escribir la dirección IP del servidor o *localhost*.

Deberías ver la página de bienvenida de Apache que se muestra en la Figura 2.



6. Para **acceder a la configuración de Apache**, debemos acudir al directorio **/etc/apache2**. Los archivos de configuración principales son el archivo **httpd.conf** y el archivo de configuración del sitio predeterminado, **000-default.conf**.

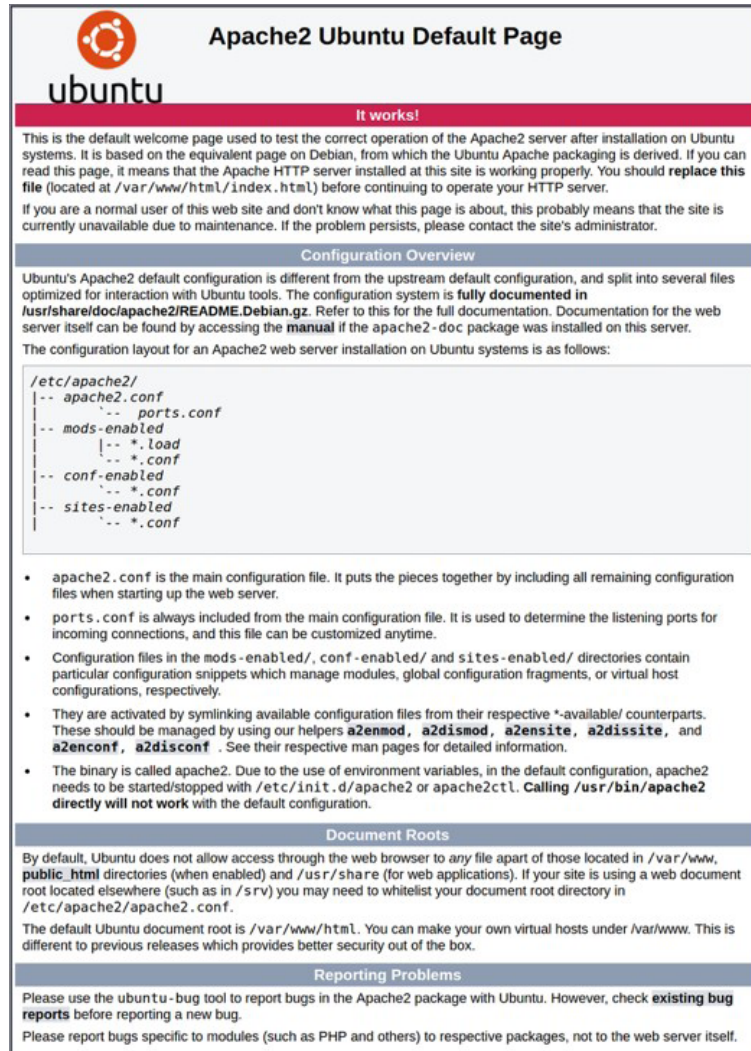


Fig.2. Página de inicio de Apache.

7. Para **probar la configuración de Apache**, vamos a ejecutar el siguiente comando:

```
sudo apache2ctl configtest
```

8. Si se recibe un mensaje de **"Syntax OK"**, la configuración es correcta.



Recuerda...

Es importante tener en cuenta que estos son solo los pasos básicos para instalar Apache en Ubuntu Server. Para una configuración más avanzada, se recomienda leer la documentación oficial de Apache y consultar recursos en línea.



2.3. Configuración avanzada de Apache

En entornos UNIX, es necesario conocer algunos comandos útiles para su ejecución correcta:

Para **detener el servidor web**, debemos escribir lo siguiente:

```
sudo systemctl stop apache2
```

Para **iniciar el servidor web** cuando no esté activo, debemos escribir lo siguiente:

```
sudo systemctl start apache2
```

Para **detener y luego iniciar el servicio de nuevo**, podemos escribir lo siguiente:

```
sudo systemctl restart apache2
```

Si solo se realizan cambios de configuración, **Apache** a menudo **puede recargarse sin cerrar conexiones**. Para hacerlo, podemos utilizar este comando:

```
sudo systemctl reload apache2
```

Por defecto, **Apache está configurado para iniciarse automáticamente** cuando el servidor lo hace. Si no es lo que se desea, podemos deshabilitar este comportamiento escribiendo lo siguiente:

```
sudo systemctl disable apache2
```

Para **volver a habilitar el servicio** de modo que se cargue en el inicio, debemos escribir lo siguiente:

```
sudo systemctl enable apache2
```

Además de lo anterior, también podemos necesitar personalizar ciertos aspectos del funcionamiento de Apache. Lo podemos hacer a través de su fichero de configuración. Este fichero puede variar en función del entorno en el que tengamos instalado Apache. Habitualmente es el fichero **httpd.conf** o **apache2.conf** (dependiendo del sistema operativo su ruta puede variar).

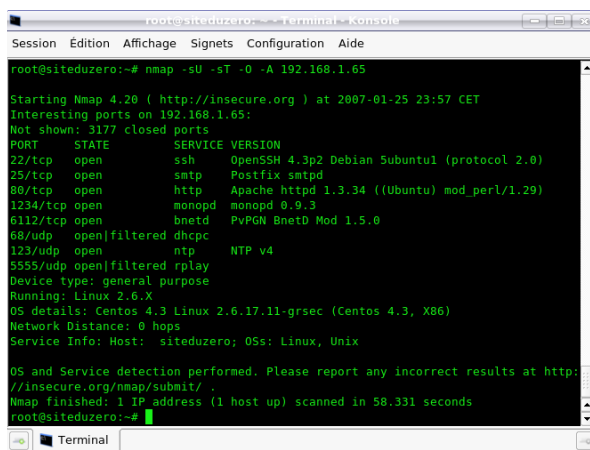


Fig.3. Configuración de Apache.



El fichero de configuración sigue un patrón sencillo. Las líneas que estén a su inicio con un carácter **#** no se ejecutarán, considerándose comentarios. Podemos modificar ciertos aspectos del funcionamiento del servidor, algunos de los más importantes son:

- **ServerRoot:** Es la ruta donde se encuentra Apache
- **ServerName:** Define el nombre y el puerto del servidor, por defecto es localhost:80
- **DocumentRoot:** es el directorio donde estarán alojados los ficheros de nuestra web. Aunque lo cambiemos, la forma de acceder a nuestro servidor, no cambia (localhost o IP), y al acceder por el navegador a nuestro servidor, se mostrarán los archivos y carpetas que haya dentro del **DocumentRoot** que tengamos definido.
- **Listen:** Define el puerto con el que Apache se comunica (por defecto el 80).



Audio 1. "¿Por qué elegir Apache como servidor web?"
<http://bit.ly/40M8bnm>



/ 3. Procesamiento de código en el servidor

Apache tan solo nos proporcionará las páginas web estáticas, con lenguajes que se ejecutarán en el cliente, como JavaScript. Para crear webs dinámicas, deberemos utilizar un lenguaje de script que se ejecute en el servidor como **PHP, JSP, ASP o Perl**.

3.1. Instalación de PHP

Para instalar PHP en un ordenador con Ubuntu Server, debemos seguir estos pasos:

1. **Abrir una terminal y actualizar la lista de paquetes** disponibles ejecutando el comando:

```
sudo apt update
```

2. **Instalar PHP y los paquetes necesarios** ejecutando el siguiente comando:

```
sudo apt install php libapache2-mod-php
```

3. Una vez instalado, debemos **verificar la versión de PHP** instalada ejecutando el siguiente comando:

```
php -v
```

4. Para **verificar que PHP está funcionando correctamente** con Apache, podemos crear un archivo PHP de prueba llamado **info.php** en el directorio **/var/www/html/** con el siguiente contenido:

```
<?php  
phpinfo();  
?>
```




5. Abrir un navegador web y escribir la dirección IP del servidor o el nombre de dominio asociado seguido de `/info.php`. Por ejemplo: `http://tu-direccion-ip/info.php`. Cuando estemos trabajando en local, la IP será `127.0.0.1` o su alias `localhost`.
6. Si se muestra **información acerca de la configuración PHP** como se muestra en la Figura 4, se muestra la configuración de PHP indicando que está funcionando correctamente.

PHP Version 7.1.33-17+ubuntu20.04.1+deb.sury.org+1	
	
System	Linux arg 5.4.0-48-generic #52-Ubuntu SMP Thu Sep 10 10:58:49 UTC 2020 x86_64
Build Date	Aug 7 2020 14:47:49
Server API	Built-in HTTP server
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php/7.1/cli
Loaded Configuration File	/etc/php/7.1/cli/php.ini
Scan this dir for additional .ini files	/etc/php/7.1/cli/conf.d
Additional .ini files parsed	/etc/php/7.1/cli/conf.d/10-mysqlnd.ini, /etc/php/7.1/cli/conf.d/10-opcache.ini, /etc/php/7.1/cli/conf.d/10-pdo.ini, /etc/php/7.1/cli/conf.d/15-xsl.ini, /etc/php/7.1/cli/conf.d/20-calendar.ini, /etc/php/7.1/cli/conf.d/20-ctype.ini, /etc/php/7.1/cli/conf.d/20-curl.ini, /etc/php/7.1/cli/conf.d/20-dom.ini, /etc/php/7.1/cli/conf.d/20-exif.ini, /etc/php/7.1/cli/conf.d/20-fileinfo.ini, /etc/php/7.1/cli/conf.d/20-ftp.ini, /etc/php/7.1/cli/conf.d/20-gd.ini, /etc/php/7.1/cli/conf.d/20-gettext.ini, /etc/php/7.1/cli/conf.d/20-iconv.ini, /etc/php/7.1/cli/conf.d/20-ison.ini, /etc/php/7.1/cli/conf.d/20-mysqli.ini, /etc/php/7.1/cli/conf.d/20-pdo_mysql.ini, /etc/php/7.1/cli/conf.d/20-phar.ini, /etc/php/7.1/cli/conf.d/20-posix.ini, /etc/php/7.1/cli/conf.d/20-readline.ini, /etc/php/7.1/cli/conf.d/20-shmop.ini, /etc/php/7.1/cli/conf.d/20-simplexml.ini, /etc/php/7.1/cli/conf.d/20-sockets.ini, /etc/php/7.1/cli/conf.d/20-sysvmsg.ini, /etc/php/7.1/cli/conf.d/20-sysvsem.ini, /etc/php/7.1/cli/conf.d/20-sysvshm.ini, /etc/php/7.1/cli/conf.d/20-tokenizer.ini, /etc/php/7.1/cli/conf.d/20-xml.ini, /etc/php/7.1/cli/conf.d/20-xmlreader.ini, /etc/php/7.1/cli/conf.d/20-xmlwriter.ini, /etc/php/7.1/cli/conf.d/20-xsl.ini, /etc/php/7.1/cli/conf.d/20-zip.ini
PHP API	20160303
PHP Extension	20160303
Zend Extension	320160303
Zend Extension Build	API320160303.NTS
PHP Extension Build	API20160303.NTS
Debug Build	no
Thread Safety	disabled
Zend Signal Handling	enabled
Zend Memory Manager	enabled
Zend Multibyte Support	disabled
IPv6 Support	enabled
DTrace Support	available, disabled
Registered PHP Streams	https, ftps, compress.zlib, php, file, glob, data, http, ftp, phar, zip
Registered Stream Socket Transports	tcp, udp, unix, udg, ssl, tls, tlsv1.0, tlsv1.1, tlsv1.2
Registered Stream Filters	zlib *, string.rot13, string.toupper, string.tolower, string.strip_tags, convert.*, consumed, dechunk, convert.iconv.*
This program makes use of the Zend Scripting Language Engine: Zend Engine v3.1.0, Copyright (c) 1998-2018 Zend Technologies with Zend OPcache v7.1.33-17+ubuntu20.04.1+deb.sury.org+1, Copyright (c) 1999-2018, by Zend Technologies	
	

Fig.4. Resultado de `phpinfo()`.

7. Para **acceder a las configuraciones de PHP**, ve al directorio `/etc/php/version` (`version` es la versión de PHP instalada) y encontrarás varios archivos de configuración como `php.ini`, `apache2.ini`, etc.



Recuerda...

Estos son solo los pasos básicos para instalar PHP en Ubuntu Server. Para una configuración más avanzada, se recomienda leer la documentación oficial y consultar recursos en línea. Además, es recomendable configurar el uso de extensiones y librerías adicionales.

/ 4. Caso práctico 1: “Configuramos Apache y PHP para mostrar los errores de código”

Planteamiento: Al igual que ocurre con Apache, la configuración de PHP se hace mediante archivos de configuración (archivos de texto en los que se especifican unos parámetros con unos valores). En el caso de PHP, principalmente, se hace mediante un único archivo. En la versión 7.4 de PHP, por ejemplo, la ruta es la siguiente:

`/etc/php/7.4/apache2/php.ini`



Nudo: La estructura del archivo de configuración de PHP es muy sencilla, las líneas que están precedidas por un ; son comentarios y no se tienen en cuenta. El resto de líneas contienen los parámetros a los que se les establece un valor mediante la forma:

```
nombre_parámetro = valor.
```

¿Serías capaz de modificar la configuración por defecto de PHP para que se muestren los errores de programación?

Desenlace: En este caso práctico debemos modificar el parámetro **display_errors**, que permite definir si queremos que los errores de nuestros programas se muestren a los usuarios a través de la web o no. Abrimos el archivo php.ini y buscamos la línea donde ponga “display_errors” y modificamos su valor, como podemos ver a continuación:

```
display_errors = On
```

A partir de este momento, los errores que se produzcan en los programas aparecerán en los archivos web, como podemos ver en la Figura 5.

```
Parse error: syntax error, unexpected 'Si' (T_VARIABLE), expecting ';' in  
C:\xampp\htdocs\program\fatalerror.php on line 3
```

Fig.5. Error de programación de PHP mostrado en el navegador.

/ 5. Sistema gestor de base de datos: Instalación y configuración

El sistema gestor de la base de datos SGBD nos permitirá **almacenar, modificar, añadir y eliminar la información** necesaria para el correcto funcionamiento de nuestras aplicaciones web.

5.1. Instalación de MySQL

Para instalar MySQL en un ordenador con **Ubuntu Server**, debemos seguir estos pasos:

1. **Abrir una terminal y actualizar la lista de paquetes** disponibles ejecutando el comando:

```
sudo apt update
```

2. **Instalar MySQL** ejecutando el siguiente comando:

```
sudo apt install mysql-server
```

3. Durante la instalación, se pedirá que establezcamos una **contraseña para el usuario root** de MySQL. Debemos asegurarnos de elegir una contraseña segura.



4. Una vez instalado, debemos iniciar el servicio de MySQL ejecutando el siguiente comando:

```
sudo systemctl start mysql
```

5. Para asegurar que **MySQL se inicie automáticamente** cada vez que el sistema se reinicie, podemos ejecutar el siguiente comando:

```
sudo systemctl enable mysql
```

6. Para **verificar si MySQL está funcionando correctamente**, debemos ejecutar el siguiente comando:

```
sudo systemctl status mysql
```

7. Para **acceder al shell de MySQL** y administrar la base de datos, debemos ejecutar el siguiente comando:

```
sudo mysql -u root -p
```

8. Una vez **dentro del shell de MySQL**, podemos ejecutar comandos SQL para crear tablas, insertar datos, etc.

5.2. Instalación de PHPMyAdmin

Para instalar **phpMyAdmin** en un ordenador con Ubuntu Server, y MySQL ya instalado debemos seguir estos pasos:

1. Ejecutar el siguiente **comando**: `sudo apt install phpmyadmin`



Fig.6. Logo de PHPMyAdmin.

2. Durante la instalación, se debe seleccionar **apache2** como **servidor web y configurar la contraseña** del usuario de phpmyadmin.

3. Una vez instalado, se debe **activar el módulo de PHP** en Apache ejecutando el siguiente comando:

```
sudo phpenmod mbstring
```

4. A continuación, debemos **reiniciar el servidor** Apache para que los cambios sean efectivos:

```
sudo systemctl restart apache2
```

5. Para acceder a phpMyAdmin, podemos **abrir un navegador web y escribir la dirección IP** del servidor seguido de **/phpmyadmin**. Por ejemplo:

```
http://tu-direccion-ip/phpmyadmin
```



6. Si se está trabajando en local, la IP será **127.0.0.1** o su alias **localhost**.
7. Por último, debemos **introducir las credenciales de usuario y contraseña** que se crearon durante la instalación para acceder a la interfaz de **phpMyAdmin**.



Recuerda...

Estos son solo los pasos básicos para instalar MySQL y PHPMyAdmin en Ubuntu Server. Para una configuración más avanzada, se recomienda leer la documentación oficial de MySQL y de PHPMyAdmin y consultar recursos en línea.

5.3. Configuración de MySQL

Por defecto, MySQL no establece contraseña para el acceso con el usuario **root**. Para acceder a MySQL como usuario **root** tan solo deberemos ejecutar el comando **mysql** como **superusuario**:



Fig.7. Logo de MySQL.

```
$ sudo mysql
```

Una vez que estamos en la consola de mysql, deberemos introducir la siguiente **instrucción**:

```
mysql > ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY 'password';
```

Debemos volver a cargar la **tabla de permisos**:

```
mysql > FLUSH PRIVILEGES;
```

A continuación, podremos **salir de la consola** de MySQL con la instrucción **exit** y volver a acceder para comprobar el acceso con la nueva contraseña con la siguiente instrucción:

```
mysql -u root -p
```

Se nos solicitará la contraseña, y se obtendrá **acceso a la consola** de MySQL. Si lo deseamos, también podemos crear usuarios nuevos para el acceso al servidor, que pueden usarse en nuestras aplicaciones. Podemos ejecutar para ello:

```
mysql > CREATE USER 'usuario'@'localhost' IDENTIFIED BY 'password';
```



Será necesario a continuación, **conceder los permisos y privilegios** al usuario que correspondan. Por ejemplo, si queremos que el usuario tenga todos los privilegios sobre todas las tablas, podemos ejecutar:

```
mysql > GRANT ALL PRIVILEGES ON *.* TO 'sammy'@'localhost' WITH GRANT OPTION;
```

5.4. Añadir un nuevo usuario a phpMyAdmin

En primer lugar, debemos **acceder a una consola de MySQL** y crear un nuevo usuario:

```
CREATE USER 'nuevo_usuario'@'localhost' IDENTIFIED BY 'contraseña';
```

A continuación, concederemos los privilegios al nuevo usuario:

```
GRANT ALL PRIVILEGES ON *.* TO 'nuevo_usuario'@'localhost';
```

Actualizamos los privilegios para que los cambios tengan efecto y salimos. En ese momento podemos acceder a phpMyAdmin con el nuevo usuario y su contraseña.



Vídeo 1. "Uso de phpMyAdmin"

<http://bit.ly/3ltPCDZ>



/ 6. Documentación de la instalación

Para documentar el proceso de instalación de un servidor web, un sistema gestor de bases de datos y un intérprete de lenguaje podemos emplear ciertas técnicas que nos facilitarán la tarea de uso y mantenimiento del sistema. Algunos de los documentos que podemos generar son:

1. **Plan de documentación:** Un plan detallado para documentar el proceso de instalación, incluyendo los **objetivos, el alcance, la estructura y los formatos de la documentación**. Recolectar toda la información necesaria para documentar el proceso de instalación, incluyendo los requerimientos previos, las configuraciones recomendadas, y los pasos necesarios para completar la instalación.
2. **Guía de instalación:** Un documento detallado que describe los **pasos necesarios para instalar el servidor web, el sistema gestor de bases de datos y el intérprete de lenguaje**, incluyendo los requerimientos previos, las configuraciones recomendadas y las dependencias necesarias.
3. **Diagrama de arquitectura:** Una **representación visual de cómo se conectan y funcionan los diferentes componentes** del sistema, incluyendo el servidor web, el sistema gestor de bases de datos y el intérprete de lenguaje.
4. **Documento sobre la configuración:** Una descripción detallada de **cómo configurar el servidor web, el sistema gestor de bases de datos y el intérprete de lenguaje**, incluyendo la configuración de seguridad, la asignación de recursos y la configuración de red.
5. **Manual de usuario:** Un documento que **describe cómo utilizar el sistema una vez instalado**, incluyendo la gestión y mantenimiento del servidor web, el sistema gestor de bases de datos y el intérprete del lenguaje.



Además de la documentación del proceso de instalación y configuración, una vez que empecemos a crear nuestros programas en el servidor, será necesario **documentar el código** para asegurar que sea fácil de entender y mantener. Algunas formas comunes de documentar el código incluyen:

1. **Comentarios:** Los comentarios son líneas o bloques de texto que se colocan en el código para proporcionar información adicional sobre lo que el código hace.
2. **Documentación de la API:** La documentación de la API es una descripción detallada de cada función, método, clase y variable del código.
3. **Documentación del sistema:** La documentación del sistema es una descripción general de cómo el código se integra en el sistema completo, incluyendo la arquitectura y las dependencias del código.
4. **Diagramas:** Los diagramas son una forma visual de representar el código y pueden incluir diagramas de clases, diagramas de flujo de trabajo y diagramas de arquitectura.
5. **Convenciones de codificación:** Es importante seguir las convenciones de codificación establecidas para el lenguaje de programación utilizado para que el código sea fácil de entender y mantener.



Fig.8. Código PHP Comentado.

/ 7. Procesamiento de código en el cliente

En este punto, ya tenemos instaladas todas las herramientas que necesitamos para ejecutar nuestras aplicaciones web en nuestro servidor. Sin embargo, si queremos editar o modificar los archivos de nuestros programas, deberemos hacer uso de editores de texto como **vi**, **nano** o **gedit**. Además de ser herramientas más complicadas de manejar, son menos potentes que un IDE.

Un **IDE** es un programa que aglutina diferentes herramientas que facilitan el proceso de desarrollo y depuración de programas. En el caso de las aplicaciones web, los IDEs pueden incluir **editores de texto** (para nuestros scripts de PHP, y las páginas HTML), **depuradores**, y **administradores de bases de datos**.

Lo ideal es instalar estas herramientas en nuestro equipo cliente, y conectar de manera remota con nuestro servidor para crear y modificar nuestras aplicaciones.

7.1. Cliente ftp

Un cliente ftp nos permitirá conectar con el servidor desde nuestro equipo, para poder descargar los archivos, modificarlos y volverlos a subir. Un cliente muy utilizado es **Filezilla**, que podemos descargar de manera gratuita desde su página oficial.



Fig.9. Logo de Filezilla.



En el siguiente enlace encontrarás más información sobre el punto anterior.

Enlaces de interés...

Visita la página del proyecto Filezilla, donde puedes descargar el software tanto para cliente como para servidor: <https://filezilla-project.org/>

7.2. Editor de texto

Un editor de texto, nos permitirá editar los archivos de nuestras aplicaciones web. Si es un editor específico de desarrollo, además proporcionará herramientas que nos faciliten la tarea del desarrollo. En el mercado existen multitud de herramientas para esta labor, cada una con sus ventajas e inconvenientes. Algunos de los más usados son: **Sublime**, **Brackets**, **Coda**, entre otros.

7.3. IDE

Un IDE es un entorno de desarrollo integrado que posee herramientas integradas para el desarrollo, prueba y depuración de programas en diferentes lenguajes y tecnologías. Normalmente, el IDE lleva integrado el cliente FTP, lo que facilita aún más la tarea, al no tener que descargar los archivos con otro programa diferente. Ejemplos son **Netbeans**, **VisualStudio**, **Coda**, entre otros. Vamos a centrarnos en uno de ellos: Netbeans.

Netbeans

NetBeans es un entorno de desarrollo integrado (IDE) que permite crear aplicaciones web mediante la edición de código fuente, la depuración y el despliegue de aplicaciones. Con NetBeans, se pueden crear proyectos web utilizando lenguajes como HTML, CSS, JavaScript y PHP. Además, el IDE proporciona herramientas para trabajar con bases de datos y servidores web, lo que facilita el despliegue de aplicaciones web complejas. Además, incluye una interfaz de usuario intuitiva y una variedad de plantillas y asistentes para ayudar a comenzar rápidamente a crear aplicaciones web.



Fig.10. Logo de Netbeans.

/ 8. Seguridad en el servidor Ubuntu

En este último punto, vamos a estudiar algunas herramientas que permiten configurar nuestro servidor desde un punto de vista de la seguridad informática.

8.1. Firewall

UFW (**Un**complicated **F**irewall) es una interfaz de línea de comandos para el **firewall de Linux iptables en Ubuntu**. UFW se diseñó para ser una herramienta fácil de usar para configurar y administrar el firewall en un sistema Ubuntu. Es una forma sencilla de permitir o bloquear el tráfico de red en un sistema Ubuntu.



Para instalar UFW en Ubuntu, se puede ejecutar el comando:

```
sudo apt-get install ufw
```

Una vez instalado UFW, se puede **activar el firewall** ejecutando el comando:

```
sudo ufw enable
```

Para **permitir el tráfico** entrante en un puerto específico, se puede usar el comando:

```
sudo ufw allow <port>
```

Por ejemplo, si se desea **permitir el tráfico entrante en el puerto 80** (HTTP) se puede usar el comando:

```
sudo ufw allow 80
```

Para **bloquear el tráfico entrante** en un puerto específico, se puede usar el comando:

```
sudo ufw deny <port>
```

Para **ver las reglas actuales en UFW**, se puede usar el comando:

```
sudo ufw status
```

UFW es una herramienta muy útil para asegurar el sistema Ubuntu, se recomienda utilizarlo para evitar problemas de seguridad y controlar el tráfico de red en el sistema.



Fig.11. Firewall para proteger el sistema de posibles ciberdelincuentes.

8.2. Servidor SSH

Para poder **desarrollar nuestras aplicaciones** desde nuestro equipo de manera remota a nuestro servidor, debemos instalar **SSH** en el servidor. Para ello, utilizaremos las siguientes instrucciones:

```
$ sudo apt-get install openssh-server
```




A continuación, deberemos **configurar el cortafuegos ufw** para que permita accesos mediante **sftp** o **ssh** a nuestro servidor:

```
$ sudo ufw allow ssh
```

Para comprobar que tenemos acceso, podemos intentar acceder por consola a través de **sftp** (desde el propio servidor, o desde otro equipo conectado a su red) conociendo su dirección IP, por ejemplo, en la dirección 192.168.1.47:

```
$ sftp usuario@192.168.1.47
```

Si hemos conseguido conectar de manera correcta, la consola cambiará por *sftp*>.

/ 9. Caso práctico 2: “Configuramos el *firewall* de nuestro sistema”

Planteamiento: El *firewall* es un *software* que va a filtrar el tráfico que accede o sale de nuestro equipo, para controlar que todos los accesos que se producen son autorizados. Nuestro servidor de aplicaciones web deberá permitir el tráfico, como mínimo a las páginas web y los sitios web seguros. Pero también debemos permitir el acceso al tráfico para poder conectarnos de forma remota.



Fig.12. Conexión remota autorizada por el firewall.

Nudo: Vamos a configurar nuestro servidor Ubuntu para que permita el tráfico entrante a nuestro servidor por HTTP, HTTPS y SSH. ¿Serías capaz de realizar la configuración adecuada?

Desenlace: Para configurar el firewall debemos seguir estos pasos:

Habilitar el *firewall* ejecutando el comando:

```
sudo ufw enable
```

Permitir el tráfico entrante en el puerto 80 (HTTP) ejecutando el comando:

```
sudo ufw allow 80
```



Permitir el tráfico entrante en el puerto 443 (HTTPS) ejecutando el comando:

```
sudo ufw allow 443
```

Permitir el tráfico entrante en el puerto 22 (SSH) ejecutando el comando:

```
sudo ufw allow 22
```

Verificar las **reglas actuales en UFW** ejecutando el comando:

```
sudo ufw status
```

Con estos pasos UFW permitirá el tráfico **HTTP, HTTPS y SSH** en tu sistema Ubuntu, esto significa que podrás navegar en internet, acceder a sitios web seguros y conectarte a tu sistema de forma remota mediante **ssh**.

/ 10. Resumen y resolución del caso práctico de la unidad

En el tema anterior nos encargamos de configurar el servidor y de instalar el sistema operativo del equipo anfitrión. En este tema hemos continuado con la realización de las instalaciones necesarias para poner a funcionar nuestro servidor de aplicaciones web.

Hemos estudiado tecnologías concretas: Apache, MySQL, phpMyAdmin y PHP, hemos aprendido a instalarlas en una máquina con Ubuntu.

Además, hemos estudiado la configuración avanzada de todos estos sistemas para que, como administradores de aplicaciones web, seamos capaces de adaptarlos a nuestros requerimientos.

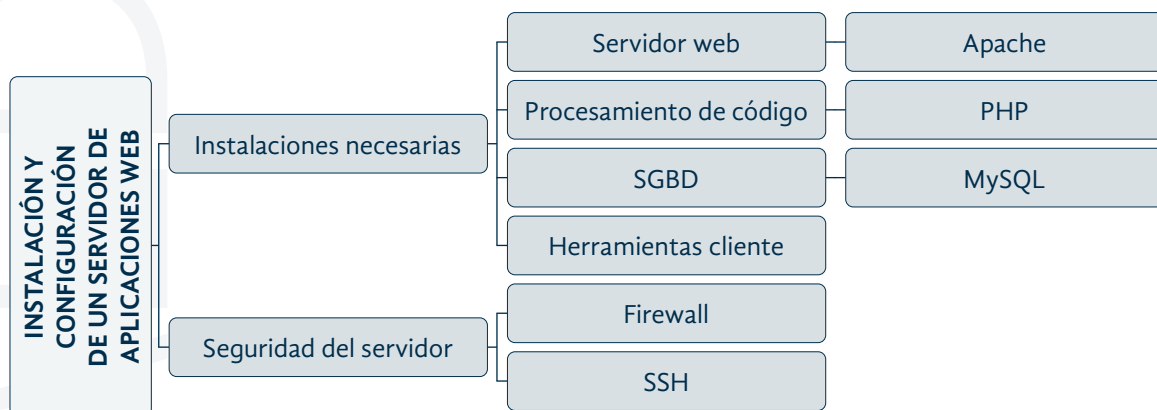


Fig.13. Esquema resumen del tema.



Resolución del caso práctico de la unidad

Debemos ir a la carpeta configurada en Apache como **DocumentRoot**, que en Ubuntu Server por defecto será: `/var/www`

Una vez en esa carpeta, se debe crear un fichero nuevo con la extensión php, por ejemplo: **holamundo.php**.

En el fichero debemos añadir el siguiente código:

```
<?php  
echo "hola mundo";  
?>
```

Tan solo nos faltaría comprobar el resultado. Abrimos un navegador web, e introducimos la IP de nuestro servidor (si estamos en local sería 127.0.0.1 o su alias localhost). El navegador debe mostrar lo siguiente:

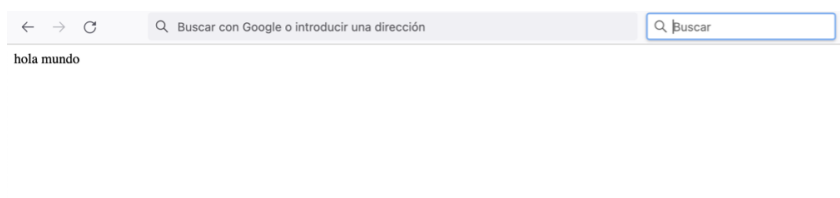


Fig.14. Resultado de la aplicación mostrado en el navegador.

/ 11. Bibliografía

Martínez, J. F. (2014). *Implantación de aplicaciones web (GRADO SUP.)*. Grupo Editorial RA-MA.

Martínez, J. F. (2015). *Implantación de aplicaciones web en entornos internet, intranet y extranet*. Grupo Editorial RA-MA.

Cabello, A. L. C. (2015). *Implantación de aplicaciones web en entornos internet, intranet y extranet*. IFCD0210. Ic Editorial.

TALLEDO SAN MIGUEL, J. V. (2015). *Implantación de aplicaciones web en entornos internet, intranet y extranet*. Ediciones Paraninfo, SA.

Official Ubuntu Documentation. (s. f.). Recuperado de: <https://help.ubuntu.com/>

Apache HTTP Server Versión 2.4 Documentación - Servidor HTTP Apache Versión 2.4. (s. f.). Recuperado de: <https://httpd.apache.org/docs/2.4/es/>

PHP: Manual de PHP - Manual. (s. f.). Recuperado de: <https://www.php.net/manual/es/>

MySQL :: MySQL Documentation. (s. f.). Recuperado de: <https://dev.mysql.com/doc/>